PROJECT SOUNDING LINE / GOVERNING DOCUMENT - PART III

PROJECT SOUNDING LINE

The Software Verification and Parallel Test Infrastructure System

PART III

Repository Policy, Codex Governance, and Release Assurance

Version 1.3 Amendment Edition - Amendment 3

Amendment purpose

Make development-workspace lifecycle, disk-capacity protection, ephemeral validation resources, worktree retirement, and cleanup receipts permanent Sounding Line law. Version 1.3 prevents reproducible execution environments from becoming accidental archives and makes storage hygiene part of phase closure and mainline acceptance.

Effective August 11, 2026 upon protected-mainline acceptance

Document ID CS-SL-P3-001 v1.3

A. Document Control and Amendment Authority

Field	Value
Program	Project Sounding Line
Subsystem	The Software Verification and Parallel Test Infrastructure System
Document	Governing Architecture and Product Requirements Document - Part III, Version 1.3 Amendment Edition (Amendment 3)
Base authority	Version 1.0, July 28, 2026, plus accepted Version 1.1 and Version 1.2 Amendment Editions.
Version	1.3
Effective date	August 11, 2026 upon acceptance through the protected Sounding Line mainline path.
Document ID	CS-SL-P3-001 v1.3
Status	Governing amendment candidate; becomes authoritative on protected-mainline acceptance. Additive and superseding only where explicitly named.
Amendment trigger	A drive-space incident in which the ForeverTreasureCompanion development root reached approximately 222 GB, the system drive fell below 3 GB free, and cleanup recovered large amounts of stale worktree dependencies, build output, rehearsal databases, and disposable validation state.
Primary objective	Preserve unique work while aggressively preventing reproducible development state from accumulating as permanent local storage.

Normative incorporation rule

Versions 1.0 through 1.2 remain governing except where this amendment adds or supersedes workspace lifecycle, cleanup, evidence-retention, Codex closure, mainline resource release, and storage-capacity requirements. Part II still owns execution mechanics; Part III v1.3 requires those mechanics to terminate in a storage-safe state.

1. Amendment Summary and Supersession Map

Existing area	Version 1.3 addition
3 Codex obligations	Adds disk-capacity preflight, heavyweight-worktree budget, task-owned storage identity, closure cleanup, and post-merge retirement obligations.
10 Release-gate classes	Adds WORKSPACE_CLOSURE as mandatory phase/mainline evidence and makes cleanup/resource disposition a gate input.
11 Mainline policy	Adds post-merge local-worktree retirement and explicit disposition of active/dormant/unique-work checkouts.
16 Evidence retention	Clarifies that successful bulky test artifacts, DB clones, browser output, and temporary storage are transient by default.
18 Test-data governance	Requires ephemeral test DB/storage to carry owner/run identity and cleanup policy.
19-20 Performance	Adds storage-capacity state and heavyweight-worktree count as workstation health/conformance metrics.
23 Machine-readable policy	Adds workspace-lifecycle policy file and enforceable thresholds.
26-27 Codex contract/report	Adds Workspace Cleanup Receipt and prohibition on

	completion with unknown temporary-resource disposition.
28 Incident policy	Adds STORAGE_CAPACITY_INCIDENT, WORKSPACE_RETENTION_INCIDENT, and CLEANUP_CONFORMANCE_INCIDENT.

2. Governing Workspace Principle and Storage Authority

Development workspaces are execution environments, not archives. Git preserves source history; governing evidence preserves proof; canonical storage preserves persistent application data. Everything else must have an explicit lifecycle and must eventually be removed, compacted, or retained by an approved exception.

Core v1.3 rule

No development phase may leave an unmanaged physical footprint behind. Reproducible state is temporary by default; unique work is preserved by default.

2.1 Preservation hierarchy

Class	Priority	Rule
Unique source / unfinished work	Highest	Preserve until pushed, merged, intentionally archived, or otherwise safely captured.
Accepted governing records / release evidence	High	Retain under Part III evidence policy and approved durable locations.
Canonical persistent application data	High	Retain only in designated development/production stores and backups.
Reproducible dependencies / builds / test state	Transient	Regenerate as needed; remove when worktree/run no longer requires them.
Unknown state	Preserve-first	Classify before deletion. Storage pressure never authorizes guessing.

2.2 Prohibited cleanup shortcuts

- git reset --hard as a storage mechanism
- git clean / git clean -fd / git clean -xdf as generic cleanup
- deleting unpushed commits
- deleting unknown untracked files
- deleting dirty worktrees merely because they are old
- deleting canonical or private data because it is large
- killing unrelated active processes to make cleanup easier

2.3 Safe uncertainty rule

If ownership, recoverability, or persistence purpose is uncertain, classify the item as QUARANTINED_REVIEW or RETAIN_UNIQUE_WORK and continue cleaning independent safe targets. One ambiguous directory never justifies stopping the entire cleanup or deleting it anyway.

3. Worktree Lifecycle Classification

Classification	Meaning	Required handling
ACTIVE_HEAVY	Current implementation/validation environment requiring dependencies/build/test state.	May retain node_modules, build output, temp DB/storage, browser state while actively needed.
ACTIVE_SOURCE_ONLY	Current source work not presently requiring a hydrated runtime.	Keep source/config; heavyweight generated state should be absent unless explicitly needed.
DORMANT_SOURCE_ONLY	Retained checkout with no active	Must be slimmed to source/config plus

	execution.	deliberately retained evidence.
COMPLETED_REMOVE	Accepted/superseded work whose meaningful commits are safely preserved.	Remove local worktree; branch history may remain remote.
RETAIN_UNIQUE_WORK	Contains meaningful staged/unstaged/untracked or unpushed unique work.	Do not remove until preservation is proven.
QUARANTINED_REVIEW	Ownership/recoverability uncertain.	Keep untouched; create review record and owner.

Default post-merge classification

A phase worktree that has reached accepted mainline, has no unique local work, and no active diagnostic need becomes COMPLETED_REMOVE immediately. Default retention: 0 days.

4. Heavyweight Worktree Budget and Disk-Capacity States

Parallel development is encouraged; parallel fully-hydrated environments are bounded. Branch count and worktree hydration are different resources. A project may have many branches without maintaining many multi-gigabyte local environments.

4.1 Default workstation heavyweight budget

Threshold	Count	Rule
Preferred maximum	4	Normal operating target.
Soft maximum	5	Before creating another, inspect disk/RAM/CPU pressure and slim dormant worktrees.
Hard maximum	6	No new ACTIVE_HEAVY worktree without closing/slimming another or recording a temporary exception.

4.2 Disk-capacity state machine

State	Free space	Required behavior
NORMAL	>= 75 GB free	Normal development and validation.
CAUTION	50-75 GB	Avoid unnecessary hydration; inspect growth trend.
RESTRICT_NEW_HEAVY_WORKTREES	30-50 GB	New heavy worktree requires explicit need and workspace audit.
CLEANUP_REQUIRED	20-30 GB	Run cleanup/reconciliation before broad new work.
STORAGE_CRITICAL	10-20 GB	Do not begin new heavyweight validation or worktrees; preserve and clean.
STOP_HEAVY_EXECUTION	< 10 GB	Only cleanup, preservation, and narrowly necessary recovery operations proceed.

Why this is a correctness rule

Git, SQLite, Prisma, Next.js, browsers, and package managers can fail in strange and destructive ways when free space collapses mid-write. Capacity preflight is therefore part of execution safety, not merely workstation housekeeping.

4.3 Capacity overrides

Thresholds may later vary by registered workstation profile, but any override must be machine-readable, documented, and accepted through Sounding Line policy. A prompt may not silently lower thresholds because the current task is inconveniently large.

5. Task-Owned Resources and Validation Ephemerality

Every mutable development/test resource must be attributable to a task, phase, or Sounding Line run and have a terminal disposition. Successful validation state is disposable unless the evidence policy explicitly requires retention.

Resource	Required ownership metadata
SQLite / test database	Run or phase ID, path, schema version, canonical-data exclusion, cleanup policy.
Object/test storage root	Run/phase owner, content class, provider namespace/root, cleanup/retention.
Browser profile/context	Run ID, browser/device class, artifact root, cleanup.
Server/process	Owner, command identity, working directory, ports, terminal state.
Build output	.next/dist/build identity, source SHA, owner, retention reason if preserved.
Validation artifacts	Evidence class, manifest, retention window, deletion authority.
Locks/leases	Owner, acquisition time, expiry/terminal release receipt.

5.1 Successful-run cleanup

- Delete successful disposable DB clones and migration rehearsal databases after evidence sealing.
- Delete successful task-owned object-storage/test roots unless explicitly retained.
- Remove ordinary Playwright traces, screenshots, videos, reports, and browser profiles after the governed evidence window.
- Release ports, process groups, leases, locks, and task-owned servers.
- Delete redundant temporary build products in dormant worktrees.
- Keep summaries, hashes, normalized receipts, and required durable release evidence.

6. Dormant Worktree Slimming and Artifact Retention

A retained worktree that is not actively executing SHALL become source-only. The ability to run npm ci, regenerate Prisma clients, rebuild Next.js output, or rerun a fixture is precisely why those bytes do not deserve permanent residency.

Artifact	Dormant-worktree rule
node_modules	Remove from DORMANT_SOURCE_ONLY unless a time-bounded explicit exception exists.
.next / dist / build	Remove when not required by active debugging or durable release evidence.
coverage / reports	Remove after evidence window; retain normalized summary if governed.
Playwright artifacts	Retain failures as needed; ordinary successes are transient.
tmp / rehearsal DB	Delete once owner/run completed and no diagnostic retention remains.
task-owned storage	Delete successful ephemeral state; preserve only explicit evidence or canonical data.
logs	Retain bounded diagnostic evidence; remove stale ordinary logs.

6.1 Shared-cache boundary

Download caches, browser binaries, immutable fixture baselines, and other read-mostly content SHOULD be shared outside worktrees when isolation remains correct. Writable application/test state may not be shared merely to save disk.

6.2 Git object maintenance

Ordinary safe Git maintenance may compact repository objects when no conflicting Git operation exists. Destructive prune/expiration intended to erase recent recoverable objects is not authorized by this amendment. Git history is not the place to reclaim the last gigabyte by gambling with unfinished work.

7. Phase Closure Workspace Cleanup Receipt

Every phase closure SHALL produce a machine-readable and human-readable Workspace Cleanup Receipt. A phase cannot claim LOCALLY COMPLETE, ACCEPTANCE COMPLETE, or equivalent closure while owned temporary resources have unknown disposition.

Receipt field	Required value
Phase / project	Formal identity and current Sounding Line authority version.
Branch / worktree	Exact local path, branch, HEAD SHA, upstream.
Remote preservation	Pushed commit status and remote parity.
Unique local work	None, preserved, or exact retained paths/reason.
Worktree classification	ACTIVE_HEAVY / ACTIVE_SOURCE_ONLY / DORMANT_SOURCE_ONLY / COMPLETED_REMOVE / RETAIN_UNIQUE_WORK / QUARANTINED_REVIEW.
Dependencies	node_modules retained/removed + reason.
Build output	.next/dist/build disposition.
Temporary databases	Paths/counts deleted, retained, canonical databases explicitly untouched.
Temporary storage	Private/community/test/backup roots deleted or retained with reason.
Browser artifacts	Reports/traces/videos/screenshots/profile disposition.

Processes/ports	Task-owned processes terminated or transferred; ports released.
Locks/leases	Validation locks and Sounding Line leases released or transferred.
Failure evidence	Retained evidence IDs, privacy class, expiry.
Disk result	Free space before/after cleanup and policy state.
Timestamp / authority	Cleanup completion time and responsible agent/owner.

Closure status rule

If product/testing scope is done but cleanup receipt is missing or fails, report: PHASE IMPLEMENTATION COMPLETE - WORKSPACE CLOSURE INCOMPLETE. Do not promote to a stronger completion status.

8. Post-Merge Worktree Retirement

Phase-level continuous integration creates a natural lifecycle: create phase worktree -> implement -> validate -> close resources -> merge to main -> retire worktree -> start next phase from fresh main. This amendment makes the retirement step mandatory by default.

8.1 Retirement gate

- Phase is accepted in protected mainline.
- Meaningful commits are safely present on the accepted remote history.
- No unique staged, unstaged, untracked, or unpushed work remains.
- No active diagnostic task requires the checkout.
- Workspace Cleanup Receipt passed.
- No active process uses the worktree.

8.2 Removal behavior

Use Git worktree-aware removal. Do not manually delete registered worktree directories first. Remote branches may remain for history; local checkout retention is not required simply because a branch exists.

8.3 Retention exception

Field	Requirement
Reason	Why local checkout must remain after accepted mainline.
Owner	Person/project responsible.
Expiry	Maximum retention date/event.
Hydration	Whether heavyweight artifacts may remain and why.
Risk	Disk/collision implications.
Revalidation	Required review at expiry.

9. Periodic Workspace Reconciliation and Storage Audit

Storage governance must detect drift before the system drive reaches emergency conditions. Audits SHALL inventory both Git registration state and physical disk usage, because stale unregistered clones and residual long-path directories can survive after Git believes a worktree is gone.

Audit	Cadence	Scope
Light audit	Every phase closure	Free-space state, heavy-worktree count, task-owned temp resources, cleanup receipt.
Full reconciliation	Weekly during heavy parallel development	Per-worktree size, hydration, unregistered clones, validation roots, caches, stale processes, Git metadata.
Emergency audit	At CLEANUP_REQUIRED or worse	Pause new heavy work; classify and reclaim safe reproducible state immediately.
Post-incident audit	After storage-capacity incident	Root-cause distribution, retained exceptions, prevention changes, baseline update.

9.1 Required audit categories

- registered worktrees and classifications
- unregistered repository clones/residual directories
- node_modules size by worktree
- build-output size by worktree
- temporary database/rehearsal size
- test/private/community storage roots
- browser/test artifacts
- Codex runtime/cache material
- Git object database
- active/orphaned processes
- disk free-space state and trend

10. Machine-Readable Workspace Lifecycle Policy

The repository SHALL encode the enforceable limits in machine-readable policy. PDF prose defines intent and authority; policy files let Sounding Line and Codex actually reject violations instead of admiring the document while the SSD fills again.

testing/workspace-lifecycle-policy.json

```
{
  "version": "1.3",
  "heavyWorktrees": {
    "preferredMaximum": 4,
    "softMaximum": 5,
    "hardMaximum": 6
  },
  "disk": {
    "normalFreeGb": 75,
    "cautionFreeGb": 50,
    "restrictNewHeavyWorktreesFreeGb": 30,
    "cleanupRequiredFreeGb": 20,
    "stopHeavyExecutionFreeGb": 10
  },
  "postMergeWorktreeRetentionDays": 0,
  "successfulValidationRetentionHours": 0,
  "failedValidationRetentionDays": 7,
  "dormantWorktree": {
    "allowNodeModules": false,
    "allowBuildOutput": false,
    "allowTemporaryDatabases": false,
    "allowBrowserArtifacts": false
  }
}
```

10.1 Required commands

Command family	Required behavior
workspace:status	Fast capacity/worktree classification report.
workspace:audit	Full storage and ownership reconciliation.
workspace:slim	Convert selected dormant checkout to source-only after safety checks.
workspace:close	Finalize task-owned resources and emit Workspace Cleanup Receipt.
workspace:retire	Verify accepted-mainline preservation and safely remove worktree.
workspace:reconcile	Audit registered/unregistered roots and policy drift.

10.2 Command safety

Workspace commands must never use broad destructive Git cleanup as an implementation shortcut. They operate through explicit classification, ownership, allowlisted generated paths, active-process checks, and fail-closed preservation when uncertainty remains.

11. Sounding Line Gate and Runtime-Conformance Integration

Workspace state becomes formal verification evidence rather than an afterthought. The protected Sounding Line decision consumes cleanup/resource-disposition evidence already required by Parts II and III and extends it with disk-capacity and worktree-lifecycle conformance.

Conformance contract	Meaning
workspace.phase-clean	Phase-owned temporary resources reconciled; receipt valid.
workspace.disk-capacity-acceptable	Disk state satisfies gate policy or approved exception.
workspace.worktree-budget	Heavyweight worktree count within policy or exception.
workspace.temp-databases-reconciled	No unidentified disposable DBs remain.
workspace.test-storage-reconciled	No unidentified test object/storage roots remain.
workspace.processes-released	Task-owned processes/ports terminated or transferred.
workspace.worktree-retirement-ready	Post-merge removal criteria are satisfied when applicable.

11.1 Gate behavior

- LOCAL_CHANGE may warn on CAUTION but must block unsafe test DB/canonical state use.
- PHASE_CLOSURE requires Workspace Cleanup Receipt.
- MAINLINE requires no unknown task-owned resources and acceptable capacity state for integration work.
- RECORD_CLOSURE normally needs no heavyweight environment; creating one without semantic need is a policy violation.
- RELEASE_CANDIDATE requires clean resource disposition and canonical-state proof.

12. Codex v1.3 Workspace Lifecycle Contract

Every future Codex task that creates or uses a worktree SHALL inherit the following contract from the effective Sounding Line authority index.

```
SOUNDING LINE v1.3 WORKSPACE CONTRACT
- Inspect disk-capacity state before creating a heavyweight worktree or broad validation environment.
- Respect the heavyweight-worktree budget; slim or retire obsolete environments before exceeding it.
- Give every temporary database, storage root, browser profile, server, port, lease, and artifact root a task/run owner.
- Preserve unique staged, unstaged, untracked, and unpushed work.
- Never use destructive Git cleanup as routine storage remediation.
- Keep dormant retained worktrees source-only.
- Clean successful disposable validation state after evidence is sealed.
- Produce a Workspace Cleanup Receipt before claiming phase closure.
- After accepted mainline and preservation proof, retire the phase worktree by default.
- Do not start the next same-project phase from an unretired old phase branch; begin from current accepted mainline.
- When free space enters CLEANUP_REQUIRED or worse, prioritize governed cleanup before new heavyweight execution.
- If an item is uncertain, preserve it, classify it, and continue cleaning independent safe targets.
```

Prompt inheritance beats prompt archaeology

Future phase prompts need not repeat this entire amendment. They must resolve the current Sounding Line authority index. The v1.3 workspace contract applies even when an older project prompt forgot to mention disk hygiene.

13. Storage and Workspace Incident Policy

Incident	Trigger	Required response
STORAGE_CAPACITY_INCIDENT	Disk reaches STORAGE_CRITICAL / STOP_HEAVY_EXECUTION or a write fails due to space exhaustion.	Stop new heavy work, preserve unique state, audit/reclaim, record root contributors and prevention.
WORKSPACE_RETENTION_INCIDENT	Completed/dormant worktrees remain hydrated beyond policy without exception.	Classify, slim/retire, identify why closure failed to retire them.
CLEANUP_CONFORMANCE_INCIDENT	Phase claimed closure with owned temp resources, DBs, processes, or storage unresolved.	Downgrade closure status, complete cleanup, repair automation/policy tests.
UNOWNED_RESOURCE_INCIDENT	Large mutable DB/storage/process/artifact cannot be tied to a task/run.	Preserve, quarantine, identify owner, prevent recurrence.
DESTRUCTIVE_CLEANUP_INCIDENT	Cleanup destroyed or threatened unique source/data through prohibited mechanism.	Stop, preserve evidence, recover, audit policy/tooling, no further automated cleanup until authorized.

13.1 Storage emergency does not weaken source safety

Even at STOP_HEAVY_EXECUTION, unique work remains higher priority than disk reclamation. The system may suspend builds and validation, but it may not convert uncertainty into permission to delete.

14. v1.3 Rollout and Enforcement Sequence

Step	Action	Result
1	Accept amendment + authority index	Add Part III v1.3 and update effective authority; future tasks inherit workspace law.
2	Add machine-readable lifecycle policy	Thresholds, classifications, retention, and receipt schema become executable.
3	Implement status/audit commands	Expose worktree hydration, disk state, temp resources, and largest contributors.

4	Implement close/slim/retire commands	Automate safe allowlisted cleanup with active-process/Git-state checks.
5	Integrate Sounding Line gates	Require Workspace Cleanup Receipt and capacity/worktree conformance.
6	Backfill current workspaces	Classify existing worktrees, slim dormant, retire completed, quarantine uncertain.
7	Establish storage baseline + trend	Record compacted root size and alert on unexplained growth.
8	Close v1.3 implementation debt	Conformance tests prove policy cannot silently regress.

15. Future Project and Phase Acceptance Additions

- Every phase declares whether it needs a heavyweight worktree and what temporary resources it will own.
- Every phase performs disk-capacity preflight before dependency installation, browser matrix, large migration rehearsal, or broad build.
- Every phase closure includes a Workspace Cleanup Receipt.
- Every post-merge phase worktree is retired immediately unless an approved retention exception exists.
- Dormant retained worktrees are source-only by default.
- Validation and browser artifacts from successful runs are transient unless evidence policy explicitly retains them.
- Test databases and storage roots are run-owned and have a cleanup policy at creation time.
- No project may normalize unbounded worktree retention or independent per-worktree caches when safe shared immutable caches exist.
- Project-status or Bridgewatch-style progress surfaces SHOULD expose workspace health when available: free disk state, active-heavy count, cleanup debt, and retained exceptions.
- The next same-project phase begins from current accepted mainline, not a pile of hydrated historical phase worktrees.

Final Part III v1.3 rule

Local development state exists to execute work, not to preserve history. Git preserves source. Governing evidence preserves proof. Canonical storage preserves data. Reproducible execution environments must remain temporary.

R. References and Amendment Closure

- Project Sounding Line Part III - Repository Policy, Codex Governance, and Release Assurance, Version 1.0, July 28, 2026.
- Project Sounding Line Part III - Version 1.1 Amendment Edition, August 10, 2026.
- Project Sounding Line Part III - Version 1.2 Amendment Edition, August 11, 2026.
- Project Sounding Line Parts I and II and their accepted amendments; Part II remains authoritative for runtime resource ownership, cleanup, and orphan recovery mechanics.
- Voyagewright Continuous Development and Mainline Integration Standard, current accepted version at implementation time.
- August 11, 2026 development-storage incident and cleanup evidence: approximately 222 GB ForeverTreasureCompanion root; system drive free space fell below 3 GB; safe cleanup recovered obsolete worktrees, dependencies/build output, rehearsal data, and Recycle Bin contents while preserving active/dirty/uncertain work.
- Current repository testing/sounding-line-authority.json, machine-readable policy, AGENTS.md, package commands, worktree inventory, and protected-mainline decision at implementation time.

All unaffected Version 1.0, Version 1.1, and Version 1.2 clauses remain governing. Version 1.3 supersedes any interpretation under which local worktrees, successful validation environments, dependency trees, build products, temporary databases, browser artifacts, or task-owned storage may accumulate indefinitely merely because they were once useful.

Appendix A. v1.3 Normative Workspace Closure Checklist

Check	Acceptance requirement

Source safety	Meaningful staged/unstaged/untracked work classified and preserved.
Remote safety	Final task commits pushed/parity verified when required.
Capacity	Disk state measured; not STOP_HEAVY_EXECUTION for ordinary closure.
Dependencies	Dormant node_modules removed or exception recorded.
Build output	Dormant build output removed or evidence reason recorded.
Databases	Task-owned disposable DBs deleted; canonical DB explicitly untouched.
Storage	Task-owned disposable object/private/community roots deleted or retained by evidence policy.
Browser artifacts	Successful transient artifacts deleted; failure evidence retained by class/expiry.
Processes	Task-owned processes terminated or transferred.
Ports / leases / locks	Released and verified.
Worktree classification	Terminal classification recorded.
Post-merge action	Retire worktree or approved retention exception recorded.
Receipt	Workspace Cleanup Receipt written and validated.

Amendment acceptance condition

Version 1.3 is fully implemented only when the authority index, machine-readable policy, commands, conformance checks, closure receipt schema, and post-merge retirement workflow are enforced by the repository rather than existing only in this PDF.